SDD aplicado a la generación de agentes

25 de octubre de 2026

Introducción

En SDD la especificación es el artefacto principal y el código del agente se deriva de ella. Para agentes esto importa más que en software clásico: su comportamiento depende de prompts, herramientas y un modelo no determinista, y sin una spec escrita nadie sabe qué debía hacer el agente ni cuándo está "terminado".

Este documento propone un flujo en seis fases, plantillas reutilizables y un ejemplo completo. Es agnóstico del framework: sirve igual con LangChain/LangGraph, el SDK de agentes de Anthropic o código propio.

Tres ideas guían todo:

- **La spec manda.** Si el agente se comporta distinto de la spec, se corrige uno de los dos de forma explícita; nunca se deja la divergencia.
- **Los criterios de aceptación son evals.** Cada requisito se traduce en casos de prueba ejecutables contra el agente real.
- **El prompt es código generado.** El system prompt se redacta a partir de la spec y se versiona con ella, no se retoca a mano sin actualizarla.

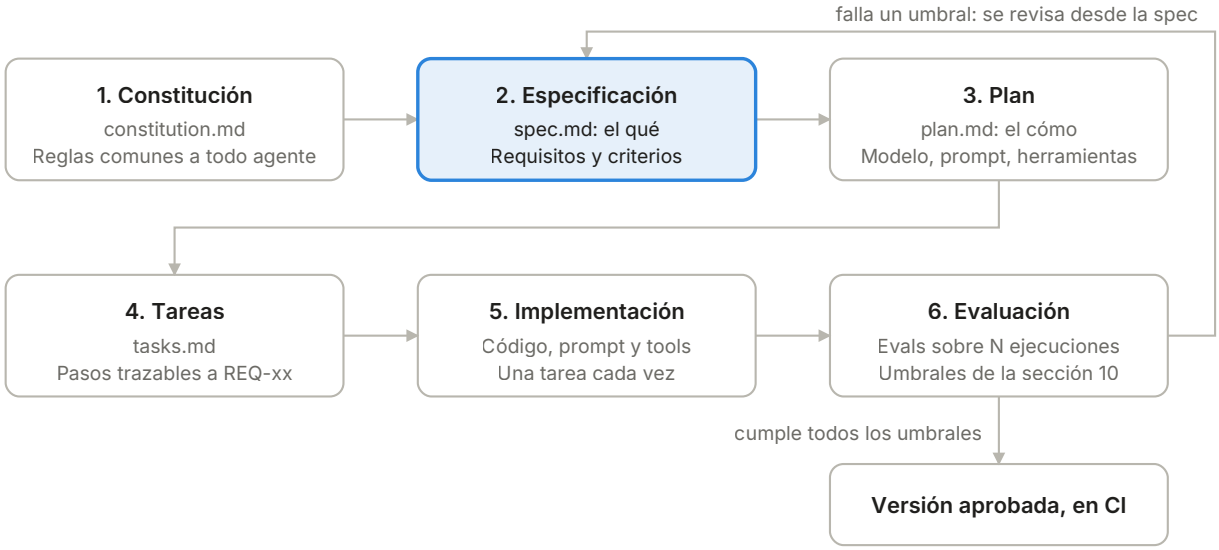
Principios de SDD adaptados a agentes

Principio SDD	Qué significa en un agente
Especificar el *qué* antes del *cómo*	Definir objetivo, usuarios, entradas, salidas y límites antes de elegir modelo, framework o herramientas.
Requisitos verificables	Cada requisito tiene un criterio medible: formato de salida, herramientas permitidas, latencia, coste por ejecución, tasa de acierto.
Contratos explícitos	Cada herramienta tiene un contrato (nombre, parámetros, errores, efectos secundarios) escrito antes de implementarla.
Límites de autonomía	La spec declara qué puede hacer el agente solo, qué requiere confirmación humana y qué tiene prohibido.
Trazabilidad	Cada tarea, prompt, herramienta y eval enlaza al requisito que la justifica (por ejemplo `REQ-03`).
Iteración controlada	Cambiar el comportamiento empieza por cambiar la spec; luego se regeneran plan, prompt y evals afectados.
No determinismo asumido	Los criterios se expresan como umbrales sobre N ejecuciones ("≥ 90 % de 50 casos"), no como igualdad exacta.

Ciclo de trabajo

El trabajo avanza en seis fases y cualquier eval que no alcance su umbral devuelve el ciclo a la especificación, no directamente al prompt.

La spec es el centro: cada fallo de eval vuelve a ella



La constitución se escribe una vez por proyecto; las fases 2 a 6 se repiten por agente y por cada cambio de comportamiento. Un asistente de código puede generar plan, tareas e implementación, pero la spec y los umbrales los aprueba una persona.

Plantilla de especificación de agente (spec.md)

La spec describe solo comportamiento observable: nada de modelos, librerías ni prompts. Si una sección no se puede rellenar, se deja como pregunta abierta en lugar de inventarla.

```
# Spec: <nombre del agente>
Versión: 0.1 · Estado: borrador | aprobada · Responsable: <persona>

## 1. Propósito
Una frase: qué problema resuelve y para quién.

## 2. Usuarios y contexto de uso
- Quién lo invoca (persona, otro agente, cron)
- Canal (CLI, API, chat, Slack)

## 3. Entradas
| Campo | Tipo | Obligatorio | Ejemplo |

## 4. Salidas
- Formato (texto, JSON con esquema, fichero)
- Esquema o ejemplo completo de salida válida

## 5. Requisitos funcionales
- REQ-01: Dado <contexto>, cuando <entrada>, el agente <comportamiento>.
- REQ-02: ...
```

```
## 6. Herramientas (capacidades)
| Herramienta | Para qué | Efectos secundarios | Requiere confirmación |

## 7. Límites de autonomía
- Permitido sin preguntar: ...
- Requiere confirmación humana: ...
- Prohibido: ...

## 8. Requisitos no funcionales
- Latencia p95: ...
- Coste máximo por ejecución: ...
- Máximo de pasos / llamadas a herramientas: ...
- Idioma, tono, privacidad de datos

## 9. Casos límite y errores
- Entrada ambigua → ...
- Herramienta falla o devuelve vacío → ...
- Petición fuera de alcance → ...

## 10. Criterios de aceptación
- AC-01 (REQ-01): <métrica> ≥ <umbral> sobre <n> casos del dataset <nombre>

## 11. Fuera de alcance

## 12. Preguntas abiertas
```

Los requisitos funcionales en formato *Dado / cuando / entonces* se traducen casi directamente en casos de eval.

Plantilla de plan técnico y desglose de tareas

El plan responde al *cómo* y cada decisión cita el requisito que la motiva. Las tareas salen del plan y son lo bastante pequeñas para que un asistente de código las implemente una a una.

```
# Plan: <nombre del agente> (basado en spec v0.1)

## Arquitectura
- Patrón: agente único con herramientas | workflow fijo | grafo (router + subagentes)
- Justificación: REQ-xx, NFR-xx

## Modelo
- Modelo principal y alternativo; motivo (coste, latencia, calidad)
- Parámetros: temperatura, máx. tokens, máx. iteraciones

## Prompt
- Secciones del system prompt y de qué apartado de la spec sale cada una
- Salida estructurada: esquema (Pydantic / JSON Schema)

## Herramientas
| Herramienta | Implementación | Contrato (entrada → salida) | Errores | REQ |

## Estado y memoria
- Qué se guarda entre pasos y entre sesiones

## Observabilidad
- Trazas por ejecución, coste y tokens, logs de llamadas a herramientas
```

```
## Riesgos
- Riesgo → mitigación
```

```
# Tareas
- [ ] T-01 Definir esquemas de entrada/salida (REQ-01, REQ-02)
- [ ] T-02 Implementar herramienta X con su contrato y tests unitarios (REQ-03)
- [ ] T-03 Redactar system prompt v1 desde spec §1, §5, §7
- [ ] T-04 Montar bucle del agente con límite de pasos (NFR-03)
- [ ] T-05 Crear dataset de evals inicial (AC-01…AC-n)
- [ ] T-06 Ejecutar evals y registrar resultados
```

Regla práctica: ninguna tarea sin requisito asociado y ningún requisito sin al menos una tarea y una eval.

Evaluación: evals como criterios de aceptación ejecutables

Un agente está terminado cuando sus evals superan los umbrales de la spec, no cuando "parece que funciona". Las evals son el equivalente a los tests de aceptación en SDD.

Tipo de eval	Qué comprueba	Cómo se puntúa	Ejemplo
Formato	La salida cumple el esquema	Validación determinista	JSON válido contra el esquema Pydantic
Uso de herramientas	Llama a las herramientas correctas, en orden razonable, sin las prohibidas	Inspección de la traza	No envía nada sin confirmación (§7)
Calidad de respuesta	Contenido correcto y completo	Comparación con referencia o LLM-as-judge con rúbrica	Respuesta cubre los 3 puntos clave
Casos límite	Comportamiento ante entradas ambiguas, errores o fuera de alcance	Rúbrica específica	Pide aclaración en vez de inventar
No funcional	Latencia, coste, número de pasos	Métricas de la traza	p95 < 20 s, coste < 0,05 €
Regresión	Lo que funcionaba sigue funcionando	Suite completa en cada cambio de spec, prompt o modelo	Ejecutar en CI

Pautas:

- Cada caso del dataset lleva el ID del criterio que cubre (`AC-01`), para medir cobertura de la spec.
- Empieza con 20–50 casos reales o realistas; añade un caso cada vez que aparezca un fallo nuevo.
- Ejecuta cada caso varias veces y mide tasas, porque el agente no es determinista.
- Si usas un LLM como juez, escribe su rúbrica en la propia spec y revisa a mano una muestra de sus veredictos.

Estructura de repositorio recomendada

Specs, prompts y evals viven en el mismo repositorio que el código y se versionan juntos, para que cada cambio de comportamiento quede en un único commit revisable.

```
mi-agente/
├── constitution.md          # reglas globales para todos los agentes
├── specs/
│   └── research-agent/
│       ├── spec.md          # el qué
│       ├── plan.md          # el cómo
│       ├── tasks.md         # tareas trazables
│       └── changelog.md     # cambios de spec y motivo
├── prompts/
│   └── research_agent.md    # generado desde spec.md
├── src/
│   ├── agents/
│   └── tools/               # una herramienta por fichero + su contrato
├── evals/
│   ├── datasets/
│   │   └── research_agent.jsonl   # casos con ID de AC
│   ├── rubrics/
│   └── run_evals.py
└── tests/                   # tests unitarios de herramientas
```

La `constitution.md` recoge lo que no cambia entre agentes: idioma por defecto, política de datos personales, acciones siempre prohibidas, límites de coste y convenciones de logging.

Ejemplo completo: agente de investigación web

Un agente que recibe una pregunta, busca en la web y devuelve un resumen con fuentes. Así quedan sus artefactos principales.

Spec (extracto)

```
## 1. Propósito
Responder preguntas factuales actuales con un resumen breve y fuentes verificables.

## 3. Entradas
| Campo | Tipo | Obligatorio | Ejemplo |
| question | str | sí | "¿Qué cambió en la última versión de X?" |
| max_sources | int | no (def. 5) | 3 |

## 4. Salidas
JSON: { "answer": str (≤ 200 palabras), "sources": [{"title", "url"}], "confidence":
"alta"|"media"|"baja" }

## 5. Requisitos funcionales
- REQ-01: Toda afirmación de "answer" está respaldada por al menos una fuente de
"sources".
- REQ-02: Si no encuentra información suficiente, lo dice y devuelve confidence = "baja".
- REQ-03: Responde en el idioma de la pregunta.

## 6. Herramientas
| web_search | buscar resultados | ninguno | no |
| fetch_page | leer una URL | ninguno | no |
```

```
## 7. Límites de autonomía
- Prohibido: inventar URLs, rellenar formularios, descargar ficheros.

## 8. No funcionales
- Máx. 8 llamadas a herramientas · p95 < 30 s

## 10. Criterios de aceptación
- AC-01 (REQ-01): ≥ 90 % de respuestas sin afirmaciones sin fuente (juez con rúbrica, 40
casos × 3 ejecuciones)
- AC-02 (REQ-02): 100 % de las 10 preguntas sin respuesta devuelven confidence = "baja"
- AC-03 (§4): 100 % de salidas válidas contra el esquema
```

Plan (decisiones clave)

- Arquitectura: agente único con dos herramientas y bucle limitado a 8 pasos (NFR §8).
- Salida estructurada con un modelo Pydantic `ResearchAnswer` (§4, AC-03).
- System prompt con cuatro bloques: rol (§1), reglas de citación (REQ-01), qué hacer sin datos (REQ-02), prohibiciones (§7).

Casos de eval (formato JSONL)

```
{"id": "c-001", "ac": ["AC-01", "AC-03"], "question": "¿Cuál es la última versión estable
de Python?", "rubric": "Cita la web oficial; versión coincide con la fuente"}
{"id": "c-031", "ac": ["AC-02"], "question": "¿Qué opinó Ana Pérez de Soria ayer en su
blog personal?", "expected": {"confidence": "baja"}}
```

Ciclo de cambio: si los usuarios piden más profundidad, primero se edita la spec (por ejemplo `answer` ≤ 400 palabras y un nuevo REQ-04), después se actualizan plan, prompt y evals, y solo entonces el código.

Llevarlo a la práctica: Claude Code + Gentle AI en la app de macOS

El ciclo de este documento se ejecuta con Claude Code en la pestaña **Code** de la app de Claude, y Gentle AI aporta las fases SDD, los subagentes y la memoria. La app lee la misma configuración que la CLI, así que todo lo que Gentle AI escribe en `~/.claude` se aplica en tus sesiones de escritorio.

Qué pone cada pieza

Pieza	Dónde vive	Papel en el ciclo
App de Claude, pestaña Code	Sesión local sobre la carpeta del repo	Donde trabajas: modos de permisos, worktrees, diff, PR
Instrucciones de Gentle AI	Secciones en `~/.claude/CLAUDE.md`	Orquestador que recorre las fases SDD
Skills SDD	`~/.claude/skills/`	Una skill por fase: `sdd-init` … `sdd-archive`
Subagentes	Herramienta Task nativa, contexto aislado por fase	Cada fase trabaja sin arrastrar el contexto de las demás

Pieza	Dónde vive	Papel en el ciclo
Engram	Servidor MCP	Memoria entre sesiones: decisiones y estado del cambio
Tu repo	`CLAUDE.md`, `.claude/skills/`, `.claude/agents/`, `openspec/`	Reglas del proyecto, skill de evals, revisor de specs

1. Instalación (una vez)

1. Instala la app de Claude para macOS, inicia sesión y abre la pestaña **Code**.
2. Instala Gentle AI y configúralo solo para Claude Code (añade `opencode` a la lista si quieres seguir usándolo en paralelo).
3. Abre una sesión nueva en la app (las sesiones abiertas no recargan la configuración) y escribe `/` en el cuadro de texto: deben aparecer las skills `sdd-*`.

```
brew install Gentleman-Programming/tap/gentle-ai

# Ver qué cambiaría sin tocar nada
gentle-ai install --dry-run --agent claude-code --preset full-gentleman

# Aplicar (hace copia de seguridad de los ficheros gestionados)
gentle-ai install --agent claude-code --preset full-gentleman

# Tras cada actualización de Gentle AI
brew upgrade gentle-ai && gentle-ai sync
```

Desde Gentle AI 3.0 el flujo por defecto es ODD y SDD hay que pedirlo de forma explícita ("usa sdd" o aceptando una propuesta SDD). Para desarrollar agentes conviene fijarlo en el `CLAUDE.md` del repo, como en el paso siguiente.

2. Preparar el repo del agente

Usa la estructura de la sección "Estructura de repositorio recomendada" y añade estos cuatro ficheros. Todos van a git.

`CLAUDE.md` del proyecto: obliga a SDD y conecta las fases de Gentle AI con tus evals.

```
# Proyecto: agentes con SDD

- Todo cambio de comportamiento de un agente usa SDD (Gentle AI), nunca ODD.
- Lee constitution.md antes de cualquier fase.
- sdd-spec rellena las 12 secciones de la plantilla de specs/; lo que falte va a
"Preguntas abiertas".
- El system prompt (prompts/) se genera desde la spec; no se edita a mano.
- sdd-verify debe ejecutar /run-evals. sdd-archive solo si se cumplen los umbrales de
§10.
- Cada commit cita los REQ o AC que toca.
```

`.claude/skills/run-evals/SKILL.md`: convierte tus evals en un comando `/run-evals`.

```
---
name: run-evals
description: Ejecuta la suite de evals del agente y compara con los umbrales de la spec
```

```
disable-model-invocation: false
allowed-tools: Bash(python evals/run_evals.py:*)
---
Ejecuta `python evals/run_evals.py --agent $ARGUMENTS --runs 3`.
Compara cada AC con su umbral en specs/$ARGUMENTS/spec.md §10.
Devuelve una tabla AC · resultado · umbral · OK/FALLA y no modifiques código.
```

`.claude/agents/spec-reviewer.md`: un revisor de solo lectura que audita la spec antes de aprobarla.

```
---
name: spec-reviewer
description: Revisa spec.md contra la plantilla SDD antes de aprobarla
tools: Read, Grep, Glob
model: opus
---
Comprueba que la spec tiene las 12 secciones, que cada REQ tiene al menos un AC
con umbral numérico y que §7 separa permitido, confirmación y prohibido.
Devuelve la lista de huecos; no edites nada.
```

Almacén de artefactos: elige modo OpenSpec o híbrido en el asistente de Gentle AI para que propuesta, spec, diseño y tareas queden en `openspec/changes/<cambio>/` dentro del repo, no solo en Engram.

3. Un cambio, fase a fase

Fase del documento	Gentle AI	Qué haces en la app	Modo de permisos
1. Constitución	`sdd-init`	Una vez por repo. Detecta stack y tests y activa TDD estricto si hay tests	Manual
Entrada a 2	`sdd-explore`, `sdd-propose`	Describe el agente o el cambio; revisa la propuesta	Plan
2. Especificación	`sdd-spec`	Lanza `spec-reviewer`, responde a sus huecos y aprueba	Plan
3. Plan	`sdd-design`	Revisa que cada decisión cite un REQ	Plan
4. Tareas	`sdd-tasks`	Comprueba una tarea por herramienta y por eval	Plan
5. Implementación	`sdd-apply`	Sigue los subagentes en el panel de tareas en segundo plano; revisa el diff	Accept edits
6. Evaluación	`sdd-verify` + `/run-evals`	Si algún AC falla, vuelve a la spec, no al prompt	Accept edits
Cierre	`sdd-archive`	Solo con todos los AC en verde; abre la PR	Manual

Paso a paso en la app:

1. **Nueva sesión local** sobre el repo, con **Worktree** activado (se crea en `.claude/worktrees/`). Una sesión por cambio, así puedes llevar varios agentes en paralelo desde la barra lateral.

2. **Cambia a modo Plan** en el selector de permisos y escribe: "Usa SDD para crear el agente de investigación web descrito en specs/research-agent/". Plan impide editar código mientras se definen propuesta, spec, diseño y tareas.
3. **Audita la spec:** "Usa el subagente spec-reviewer sobre la spec". Responde a los huecos y aprueba solo cuando §7 y §10 estén completos.
4. **Pasa a Accept edits** y pide que continúe con `sdd-apply`. Gentle AI delega cada tarea en un subagente; los ves en el panel de tareas en segundo plano.
5. **Revisa el diff** desde el contador de líneas de la tarjeta Changes. Comenta líneas concretas y envíalas juntas con Cmd+Enter; Claude aplica los cambios como un diff nuevo. Ejecuta `/code-review` antes de seguir.
6. **Evalúa:** `/run-evals research-agent`. Si un AC falla, pide "vuelve a sdd-spec: el AC-02 falla por..." en vez de retocar el prompt a mano.
7. **Cierra:** con todo en verde, `sdd-archive` actualiza `changelog.md`. Abre la PR desde la sesión y activa **Auto-fix** para que Claude atienda checks de CI y comentarios; **Merge when ready** activa el auto-merge de GitHub. Requiere `gh` instalado y autenticado.

4. Mantener los agentes en verde

- **Regresión nocturna:** crea una tarea programada de Desktop que ejecute `/run-evals` sobre cada agente y te avise si un AC baja del umbral.
- **Revisión adversarial opcional:** el modo de revisión de Gentle AI (`gentle-ai review mode enable`) añade revisores de solo lectura sobre el cambio terminado; nunca aprueba la entrega por ti.
- **Memoria:** Engram guarda decisiones y estado del cambio; si cierras la app a mitad de un cambio, la sesión siguiente lo retoma.
- **Si `/gentle-sdd-*` no aparece:** la documentación de Gentle AI se contradice sobre los slash commands en Claude Code. Pedir "usa sdd" en lenguaje natural activa el mismo flujo.

Fuentes: Claude Code en Desktop · directorio .claude · tareas programadas · agentes de Gentle AI · wiki de Gentle AI · Gentle AI 3.0.0

Skills en el vibe coding

Una skill es una carpeta con un `SKILL.md` que empaqueta un procedimiento: Claude solo ve su descripción y carga el resto cuando la necesita. En el vibe coding convierte los prompts que repites en flujos reutilizables y consistentes; de hecho, las fases SDD de Gentle AI son skills (Skills en Claude Code).

Crea una skill cuando pegues las mismas instrucciones por tercera vez, o cuando una sección de `CLAUDE.md` se haya convertido en un procedimiento paso a paso.

Qué es una skill y qué no

Mecanismo	Cuándo se carga	Úsalo para
`CLAUDE.md`	Siempre, entero, en cada sesión	Hechos y convenciones cortas: "usamos SDD", "Python 3.12"

Mecanismo	Cuándo se carga	Úsalo para
Skill	Descripción siempre; cuerpo solo al invocarla	Procedimientos y material de referencia: "cómo añadir una herramienta al agente"
Subagente	Cuando se le delega; contexto propio	Trabajo aislado con otro prompt, herramientas o modelo: revisar una spec
Servidor MCP	Sus herramientas, siempre disponibles	Conectar sistemas externos: Engram, GitHub, una base de datos
Hook	Automático en eventos (antes o después de una herramienta)	Reglas que deben cumplirse siempre, sin depender del modelo

Se combinan: una skill con `context: fork` se ejecuta dentro de un subagente, y un subagente puede precargar skills con su campo `skills`.

Dónde viven

Alcance	Ruta	Para qué
Personal	`~/.claude/skills/<nombre>/SKILL.md`	Tus hábitos en todos los proyectos (aquí instala Gentle AI las suyas)
Proyecto	`.claude/skills/<nombre>/SKILL.md`	Procedimientos del repo; van a git y los comparte el equipo
Plugin	`<plugin>/skills/<nombre>/SKILL.md`	Paquetes instalables; se invocan como `/plugin:skill`

Claude Code trae además skills integradas como `/code-review` o `/debug`. Los ficheros antiguos `.claude/commands/*.md` siguen funcionando.

Anatomía de una skill

Ejemplo para el flujo de este documento: añadir una herramienta a un agente respetando su contrato y su spec.

```
---
name: add-tool
description: Añade una herramienta a un agente con contrato, tests y actualización de la
spec. Úsala cuando el usuario pida una nueva tool o capacidad.
disable-model-invocation: true
allowed-tools: Bash(pytest *)
---

## Spec actual
!`cat specs/$0/spec.md`

## Instrucciones
1. Añade la herramienta $1 a la tabla §6 de la spec y, si tiene efectos secundarios, a
§7.
2. Crea src/tools/$1.py con docstring de contrato: entrada, salida, errores.
3. Escribe tests en tests/test_$1.py y ejecútalos con pytest.
```

```
4. Añade al menos un caso de eval que la use, con su ID de AC.
5. No toques el system prompt: se regenera desde la spec.
```

Con `/add-tool research-agent fetch_page`, `$0` y `$1` toman los dos argumentos. La línea `!`cat …`` se ejecuta antes de que Claude lea la skill, así que las instrucciones llegan con la spec real.

Para material largo, separa en ficheros dentro de la carpeta y enlázalos desde `SKILL.md`; solo se cargan cuando hacen falta:

```
add-tool/
├── SKILL.md            # resumen y pasos
├── contrato.md         # plantilla de contrato de herramienta
└── scripts/check.py    # se ejecuta, no se carga en contexto
```

Cómo se invocan

- **Manual:** escribe `/nombre-skill` y sus argumentos en el cuadro de texto de la pestaña Code.
- **Automática:** Claude compara tu petición con la `description` y la carga si encaja. Por eso la descripción decide si la skill se usa.

Campo del frontmatter	Efecto
`description`	Qué hace y cuándo usarla; pon primero el caso principal (se corta a 1.536 caracteres)
`disable-model-invocation: true`	Solo la invocas tú. Úsalo en todo lo que tenga efectos: desplegar, publicar, borrar
`user-invocable: false`	Solo la usa Claude. Para conocimiento de fondo, como las convenciones del framework
`allowed-tools`	Herramientas que no piden permiso mientras la skill está activa
`context: fork` + `agent`	La ejecuta un subagente sin tu historial; las instrucciones deben ser autosuficientes
`paths`	Solo se autocarga al trabajar con ficheros que encajan con esos patrones

Skills útiles para desarrollar agentes

Skill	Qué hace	Invocación
`new-agent-spec`	Crea `specs/<agente>/spec.md` desde la plantilla y pregunta por los huecos	Manual
`add-tool`	Herramienta + contrato + tests + spec + eval (ejemplo de arriba)	Manual
`run-evals`	Ejecuta la suite y compara con los umbrales de §10	Manual
`eval-from-bug`	Convierte un fallo observado en un caso de eval con su AC	Manual

Skill	Qué hace	Invocación
`prompt-from-spec`	Regenera el system prompt a partir de §1, §5 y §7	Automática, con `paths: specs/**`
`framework-conventions`	Convenciones de tu stack (LangChain, SDK de agentes…)	Solo Claude (`user-invocable: false`)

Buenas prácticas

- **Una skill, un trabajo.** Si la descripción necesita un "y además", son dos skills.
- **Cuerpo corto:** menos de 500 líneas. Una vez cargada, ocupa contexto en los turnos siguientes; di qué hacer, no por qué.
- **Efectos secundarios, siempre manuales:** con `disable-model-invocation: true` Claude no los dispara por su cuenta.
- **Pruébala como pruebas un agente:** lanza peticiones realistas en sesiones nuevas, con y sin la skill, y comprueba que se activa cuando debe y que el resultado mejora.
- **Versiónala con el repo:** las skills de proyecto son parte del proceso SDD; un cambio de procedimiento se revisa en una PR como cualquier otro.

Antipatrones y checklist final

Antipatrón	Síntoma	Corrección
Prompt-first	Se itera el prompt durante días y nadie sabe qué debe hacer el agente	Escribir la spec y derivar el prompt de ella
Spec de deseos	Requisitos como "debe ser útil y preciso"	Reescribir como Dado / cuando / entonces con umbral
Spec congelada	El código cambia y la spec no	Toda PR que cambie comportamiento toca `spec.md` y `changelog.md`
Herramientas sin contrato	Errores silenciosos o salidas inesperadas de una tool	Documentar entrada, salida y errores; tests unitarios
Autonomía implícita	El agente hace acciones que nadie aprobó	Sección §7 obligatoria y comprobada en evals de traza
Eval por intuición	"Lo probé con tres preguntas y va bien"	Dataset versionado y umbrales sobre varias ejecuciones
Sobrearquitectura	Multiagente para un problema de una herramienta	Empezar con el patrón más simple que cumpla la spec

Checklist antes de dar un agente por terminado

- ☐ La spec está aprobada y no tiene preguntas abiertas bloqueantes
- ☐ Cada requisito tiene ID, al menos una tarea y al menos una eval
- ☐ Cada herramienta tiene contrato y tests unitarios

- ☐ Los límites de autonomía están en el prompt y se verifican en las trazas
- ☐ El system prompt está versionado junto a la spec que lo origina
- ☐ Las evals superan todos los umbrales de la sección 10
- ☐ Latencia, coste y número de pasos están dentro de lo especificado
- ☐ Las evals de regresión corren automáticamente en cada cambio